

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №6
«Настройка GitHub Actions для автоматического
развёртывания бэкенд-приложения»

Выполнила:

Олейник Полина

Группа 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Необходимо настроить автодеплой (с триггером на обновление кода в репозитории, на определённой ветке) для приложения на удалённый сервер с использованием GitHub Actions или GitLab CI.

Тема проекта: вариант 1 – приложение для бронирования столиков в ресторанах. Настраивался автодеплой уже развёрнутого в ЛР4 приложения на сервер в Yandex Cloud, с использованием GitHub Actions.

Ход работы

1. Подготовка ключа для доступа CI к серверу

Сначала я сгенерировала отдельную пару SSH-ключей специально для автодеплойа – не ту, которой сама захожу на сервер:

```
ssh-keygen -t ed25519 -f C:\Users\User\.ssh\deploy-key -N ''
```

Публичный ключ (deploy-key.pub) добавила на сервер в список разрешённых:

```
# на сервере
nano ~/.ssh/authorized_keys
# вставила содержимое deploy-key.pub отдельной строкой
```

Проверила, что новым ключом действительно можно зайти на сервер без пароля:

```
ssh -i C:\Users\User\.ssh\deploy-key polinaoleynik@89.169.170.37
```

```
PS C:\Users\User> ssh -i C:\Users\User\.ssh\deploy-key polinaoleynik@89.169.170.37
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-139-generic x86_64)
```

Подключение к серверу новым ключом прошло успешно

2. Секреты репозитория на GitHub

В настройках своего форка (Settings → Secrets and variables → Actions) добавила три секрета: SERVER_HOST (IP сервера), SERVER_USER (имя пользователя) и SSH_PRIVATE_KEY (содержимое приватного ключа deploy-key целиком, вместе со строками BEGIN и END). Эти значения не попадают в код и не видны в логах запусков.

Repository secrets			New repository secret
Name ↕		Last updated	
SERVER_HOST		now	 
SERVER_USER		now	 
SSH_PRIVATE_KEY		1 minute ago	 

Три секрета добавлены в репозиторий

3. Файл workflow

Создала файл `.github/workflows/deploy.yml` в корне репозитория со следующим содержимым:

```
name: Deploy to server

on:
  push:
    branches: [deploy]
    paths:
      - 'BP2.1/Олейник Полина/labs/lab3/**'

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${ secrets.SERVER_HOST }
          username: ${ secrets.SERVER_USER }
          key: ${ secrets.SSH_PRIVATE_KEY }
          script: |
            set -e
            cd "app/BP2.1/Олейник Полина/labs/lab3"
            git pull origin deploy
            docker compose up -d --build
            docker compose ps
```

Триггер настроен так, чтобы workflow запускался только при пуше в ветку `deploy` и только если изменения затронули папку с моим проектом – это защита от того, чтобы автодеплой случайно не сработал от чужого пуша в общем репозитории курса.

4. Ветка `deploy` и первый пуш

Код приложения на момент этой работы лежал в ветке `lab3`, а не в `main`, поэтому новую ветку создала именно от неё:

```
git checkout lab3
git pull
git checkout -b deploy
mkdir .github\workflows -Force
copy deploy.yml .github\workflows\deploy.yml
git add .github\workflows\deploy.yml
git commit -m "ci: add auto-deploy workflow"
git push -u origin deploy
```

5. Переключение сервера на новую ветку

На сервере подтянула новую ветку и переключилась на неё, чтобы дальнейшие `git pull` из workflow применялись именно к `deploy`:

```
cd "app/БР2.1/Олейник Полина/labs/lab3"
git fetch origin
git checkout deploy
```

```
polina@leynik@compute-vm-2-4-20-ssd-1798363084242:~/app/БР2.1/Олейник Полина/labs/lab3$ git fetch origin
remote: Enumerating objects: 12, done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 12 (delta 1), reused 12 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (12/12), 1.71 MiB | 5.41 MiB/s, done.
From https://github.com/Fieria/ITMO-ACS-Backend-2026-C
 * [new branch]      deploy -> origin/deploy
 * [new branch]      lab4   -> origin/lab4
polina@leynik@compute-vm-2-4-20-ssd-1798363084242:~/app/БР2.1/Олейник Полина/labs/lab3$ git checkout deploy
M       320\221\320\24082.1\420\236\320\273\320\265\320\271\320\275\320\270\320\272  \320\237\320\276\320\273\320\270\3
20\275\320\260\labs/lab3/docker-compose.yml
branch 'deploy' set up to track 'origin/deploy'.
Switched to a new branch 'deploy'
```

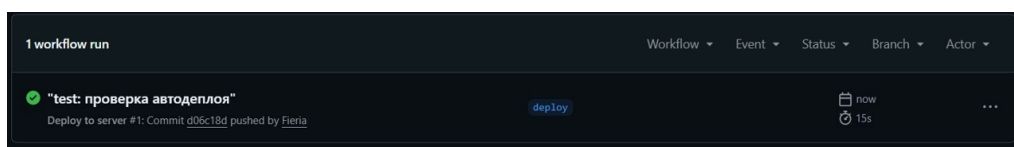
Сервер увидел новую ветку `deploy` и переключился на неё

6. Тестовый пуш и проверка автодепоя

Чтобы реально проверить срабатывание триггера (первый пуш менял только сам файл workflow и не подходил под фильтр `paths`), внесла небольшое изменение в код внутри своей папки и запустила его в ветку `deploy`:

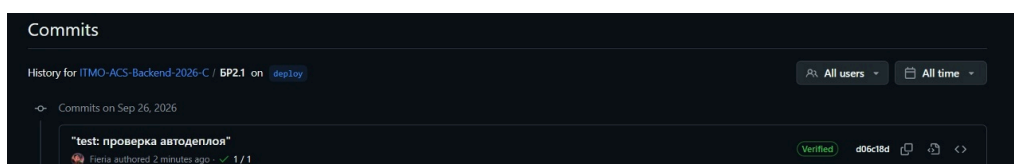
```
git add .
git commit -m "test: проверка автодеплойа"
git push
```

Во вкладке Actions на GitHub сразу появился запуск workflow «Deploy to server», и через 15 секунд он завершился зелёной галочкой:



Workflow «Deploy to server» отработал успешно

В истории коммитов ветки `deploy` виден именно этот коммит, помеченный как `Verified`:



Коммит "test: проверка автодеплов" в ветке deploy

7. Проверка на сервере

Зашла на сервер и убедилась, что код обновился автоматически, без единой ручной команды с моей стороны, а все шесть контейнеров продолжают работать:

```
git log -1
docker compose ps
```

```
commit d06c18ddd7dfdd8700fd62d74cc1555bb38dabc (HEAD -> deploy, origin/deploy)
Author: Fieria <104035058+Fieria@users.noreply.github.com>
Date: Sat Sep 26 02:14:47 2026 +0300

"test: проверка автодеплойа"
polinaoleynik@compute-vw-2-4-20-ssd-1790363084242:~/app/БП2.1/Олейник Полина/labs/lab3$ docker compose ps
NAME                IMAGE                                COMMAND
SERVICE            CREATED      STATUS      PORTS
lab3-auth-1          sha256:27d791803a67135e98310bd1f7d5eac24f4b232a6c821105080cfa173354bf7  "docker-entrypoint.s..." auth        2 hours ago Up 2
lab3-auth-1          3001/tcp
lab3-bookings-1      sha256:8db1f07576c2bbf6d6ff08234bb40248727afe88253ba9ffd74d4bc26612b07b  "docker-entrypoint.s..." bookings  2 hours ago Up 2
lab3-bookings-1      3003/tcp
lab3-db-1            postgres:17                          "docker-entrypoint.s..." db        2 hours ago Up 2
lab3-db-1            0.0.0.0:15433->5432/tcp, [::]:15433->5432/tcp
lab3-gateway-1       sha256:305b69d6aceb920e8d537c37eda0991287d62d4566b6d67d55aebae45645675e  "docker-entrypoint.s..." gateway  2 hours ago Up 2
lab3-gateway-1       127.0.0.1:8080->8080/tcp
lab3-rabbitmq-1      rabbitmq:3-management               "docker-entrypoint.s..." rabbitmq  2 hours ago Up 2
lab3-rabbitmq-1      4369/tcp, 5671/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:5673->5672/tcp, [::]:5673->5672/tcp, 0.0.0.0:15673->15672/tcp, [::]:
lab3-restaurants-1  sha256:15788a032f68f442d717420961e3627125859c6424642b3a6f2b1578e2562294  "docker-entrypoint.s..." restaurants 2 hours ago Up 2
lab3-restaurants-1  3002/tcp
```

На сервере — тот же коммит, что и в GitHub, все контейнеры работают

Вывод `git log -1` на сервере показал именно тот коммит `test: проверка автодеплойа`, который я перед этим запустила с компьютера — значит, GitHub Actions действительно подключился к серверу и сам выполнил `git pull` и перезапуск контейнеров.

Вывод

В ходе работы я настроила автоматическое развёртывание приложения на удалённый сервер при помощи GitHub Actions. Создала отдельный SSH-ключ для CI, добавила его на сервер и сохранила секреты доступа (адрес сервера, пользователя, приватный ключ) в настройках репозитория. Написала workflow-файл, который запускается при пуше в ветку `deploy` и только при изменениях в моей папке проекта, подключается к серверу по SSH, подтягивает новый код и пересобирает контейнеры командой `docker compose up -d --build`. Проверила результат тестовым пушем: во вкладке Actions запуск завершился успешно за 15 секунд, а на самом сервере оказался тот же коммит, что был отправлен с компьютера, и все шесть контейнеров продолжили работать без сбоев. Автодеплой работает: теперь для обновления приложения на сервере достаточно запустить изменения в ветку `deploy`, без ручного подключения по SSH и выполнения команд.